

TRƯỜNG ĐẠI HỌC BÁCH KHOA
THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO THỰC TẬP

Công ty TNHH Amazon Web Services Việt Nam

Workforce Bootcamp - First Cloud AI Journey

Thông tin sinh viên:

Họ và tên: Thái Đức Khang
Lớp: CC23KHM
MSSV: 2352500
Chuyên ngành: Khoa học Máy tính
Email: khang.thaiduc@hcmut.edu.vn
Số điện thoại: 0917068787
Thời gian: 6/6/2026 – 15/08/2026

TP. Hồ Chí Minh, Ngày 31 tháng 7 năm 2026

Mục lục

1	Nhật ký công việc	3
2	Mô tả dự án	16
2.1	Tổng quan dự án	16
2.2	Mục tiêu dự án	16
2.3	Lập kế hoạch	17
2.3.1	Giai đoạn 1: Phát triển ứng dụng cốt lõi	17
2.3.2	Giai đoạn 2: Xác thực và Phân quyền	18
2.3.3	Giai đoạn 3: Tính năng Mượn, Trả, Đặt trước và Thông báo	18
2.3.4	Giai đoạn 4: Đóng gói Container và CI/CD	18
2.3.5	Giai đoạn 5: Triển khai và Giám sát trên AWS	19
2.4	Ngăn xếp Công nghệ	19
2.5	Thiết kế và Kiến trúc	19
2.6	Phân tích Kiến trúc	20
2.6.1	Phát triển và Quản lý Mã nguồn	20
2.6.2	Quy trình CI/CD với GitHub Actions	20
2.6.3	Lưu trữ Hình ảnh Container với Amazon ECR	21
2.6.4	Triển khai Ứng dụng với AWS Elastic Beanstalk	21
2.6.5	Giao tiếp giữa Frontend và Backend	21
2.6.6	Tầng Cơ sở dữ liệu với PostgreSQL	22
2.6.7	Giám sát với Amazon CloudWatch	22
2.6.8	Vận hành Đám mây với AWS CloudShell	22
2.7	Quy trình Triển khai	23
2.8	Thiết kế Bảo mật và Kiểm soát Truy cập	23
2.9	Cân nhắc về Khả năng Mở rộng và Khả năng Bảo trì	24
2.10	Kết quả Dự kiến	24
2.11	Kết luận	24
3	Các bài Blog	25
4	Các sự kiện đã tham gia	25
5	Hội thảo: Triển khai CloudLibrary lên AWS	27
5.1	Tổng quan	27
5.2	Điều kiện tiên quyết	28
5.3	Kiến trúc triển khai	30
5.4	Quy trình triển khai	31
5.4.1	Bước 1: Kiểm tra Cấu trúc Dự án	31
5.4.2	Bước 2: Kiểm thử Ứng dụng Cục bộ (Local)	31

5.4.3	Bước 3: Kiểm chứng Backend	32
5.4.4	Bước 4: Build Frontend	33
5.4.5	Bước 5: Tạo các kho lưu trữ (repository) Amazon ECR	33
5.4.6	Bước 6: Cấu hình Xác thực GitHub với AWS	34
5.4.7	Bước 7: Cấu hình Elastic Beanstalk	34
5.4.8	Bước 8: Cấu hình EC2 Instance Role	35
5.4.9	Bước 9: Cấu hình Workflow GitHub Actions	36
5.4.10	Bước 10: Push Mã nguồn lên GitHub	37
5.4.11	Bước 11: Build và Push Docker Images	37
5.4.12	Bước 12: Tạo Gói Triển khai (Deployment Bundle)	38
5.4.13	Bước 13: Triển khai Ứng dụng	38
5.4.14	Bước 14: Xác minh Triển khai	39
5.4.15	Bước 15: Giám sát Ứng dụng	40
5.4.16	Bước 16: Khắc phục sự cố (Troubleshoot) khi Triển khai Thất bại . . .	41
5.4.17	Bước 17: Khôi phục (Roll Back) về Phiên bản Ổn định	42
5.4.18	Bước 18: Dọn dẹp Tài nguyên Không sử dụng	42
5.5	Kết quả Triển khai	43
6	Tự đánh giá	45
7	Chia sẻ và Phản hồi	47
7.1	Đánh giá Chung	47
7.2	Các câu hỏi bổ sung	48
7.3	Đề xuất & Kỳ vọng	48

1 Nhật ký công việc

Bản ghi công việc này ghi lại những đóng góp cá nhân của tôi vào dự án **CloudLibrary – Hệ thống mượn sách trên AWS** từ **15/06/2026 đến 30/07/2026**. Trong nhóm ba thành viên, Thành viên A chịu trách nhiệm phát triển frontend, Thành viên B chịu trách nhiệm phát triển backend, trong khi tôi đảm nhận phần công việc còn lại bao gồm tích hợp hệ thống, Docker, quản lý quy trình làm việc trên GitHub, hạ tầng AWS, CI/CD, giám sát, khắc phục sự cố triển khai, kiểm thử và tài liệu kỹ thuật.

Nhật ký công việc Tuần 1

Thời gian: 15/06/2026 – 19/06/2026

Mục tiêu Tuần 1

- Hiểu rõ các yêu cầu của dự án CloudLibrary và xác định trách nhiệm của từng thành viên trong nhóm.
- Nghiên cứu các dịch vụ AWS cần thiết để triển khai một ứng dụng full-stack được đóng gói bằng container.
- Chuẩn bị chiến lược quản lý mã nguồn và kiến trúc triển khai AWS ban đầu.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Đánh giá các yêu cầu thực tập, kế hoạch dự án và cấu trúc báo cáo dự kiến. Thảo luận về phạm vi dự án và phân chia nhóm thành các mảng frontend, backend và Cloud/DevOps.	15/06/2026	15/06/2026	First Cloud AI Journey
3	Nghiên cứu vai trò của IAM, Amazon ECR, Elastic Beanstalk, Amazon S3, CloudWatch, EC2 và AWS CLI trong quy trình triển khai ứng dụng dạng container.	16/06/2026	16/06/2026	AWS Overview
4	Xem xét mã nguồn React frontend và Flask backend hiện có. Xác định đường dẫn API, biến môi trường, kết nối cơ sở dữ liệu, điểm kiểm tra trạng thái và các điểm tích hợp.	17/06/2026	17/06/2026	Docker Compose
5	Chuẩn bị cấu trúc kho chứa Git và chiến lược nhánh, bao gồm develop, develop_2.0 và các nhánh tính năng. Xác định các quy tắc commit, merge và tích hợp.	18/06/2026	18/06/2026	GitHub documentation
6	Lên phác thảo kiến trúc triển khai ban đầu và quy ước đặt tên tài nguyên. Lựa chọn ap-southeast-2 làm vùng AWS và lập kế hoạch luồng CI/CD.	19/06/2026	19/06/2026	AWS Regions

Thành tựu Tuần 1

- Hiểu rõ tổng quan kiến trúc và phạm vi dự án CloudLibrary.
- Xác định rõ ràng sự phân chia trách nhiệm cho nhóm ba thành viên.
- Nhận diện được các dịch vụ AWS cần thiết cho việc triển khai và giám sát.
- Thiết lập chiến lược nhánh Git và tổ chức kho chứa mã nguồn.
- Tạo ra phiên bản đầu tiên của kiến trúc triển khai trên AWS.

Nhật ký công việc Tuần 2

Thời gian: 22/06/2026 – 26/06/2026

Mục tiêu Tuần 2

- Tích hợp các dịch vụ frontend, backend và PostgreSQL trong môi trường cục bộ.
- Đóng gói ứng dụng bằng Docker và Docker Compose.
- Xác minh rằng toàn bộ ứng dụng có thể chạy ổn định cục bộ trước khi triển khai lên AWS.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Tạo kế hoạch tích hợp cục bộ và chuẩn bị <code>.env.example</code> . Xác định các biến môi trường cho PostgreSQL, Flask, JWT, ghi log ứng dụng và định tuyến API.	22/06/2026	22/06/2026	Docker env variables
3	Làm việc với backend member để chuẩn bị container backend, lệnh Gunicorn, cài đặt thư viện phụ thuộc và endpoint <code>/health</code> .	23/06/2026	23/06/2026	Gunicorn docs
4	Làm việc với frontend member để chuẩn bị bản build React, cấu hình Nginx, phân phối tài nguyên tĩnh và reverse proxy từ <code>/api</code> .	24/06/2026	24/06/2026	Nginx proxy
5	Tạo và tinh chỉnh <code>docker-compose.yml</code> với ba dịch vụ: <code>db</code> , <code>backend</code> và <code>frontend</code> . Thêm phụ thuộc, health checks, mạng và volume.	25/06/2026	25/06/2026	Compose reference
6	Chạy toàn bộ ứng dụng cục bộ, kiểm tra xác thực và API, sửa lỗi CORS và đường dẫn API, đồng thời lập tài liệu hướng dẫn lệnh khởi động.	26/06/2026	26/06/2026	Docker Compose up

Thành tựu Tuần 2

- Tích hợp thành công các dịch vụ frontend, backend và PostgreSQL ở môi trường cục bộ.
- Xây dựng thành công môi trường Docker Compose cho toàn bộ hệ thống.
- Cấu hình Nginx làm máy chủ web frontend và reverse proxy cho API.
- Thêm tính năng kiểm tra trạng thái và kiểm soát thứ tự khởi động dịch vụ.

- Kiểm chứng luồng hoạt động chính của ứng dụng trước khi tiến hành triển khai lên AWS.

Nhật ký công việc Tuần 3

Thời gian: 29/06/2026 – 03/07/2026

Mục tiêu Tuần 3

- Xây dựng quy trình tích hợp liên tục cho dự án.
- Tự động kiểm tra các bài test backend và quá trình build production của frontend.
- Chuẩn hóa quá trình tích hợp trước khi triển khai lên AWS.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Tạo quy trình CI GitHub Actions ban đầu và cấu hình triggers cho các nhánh dự án. Thêm checkout mã nguồn và thiết lập môi trường Python/Node.js.	29/06/2026	29/06/2026	GitHub Actions
3	Thêm cài đặt thư viện phụ thuộc backend và tự động chạy pytest. Xem xét quá trình phát hiện test và đường dẫn import.	30/06/2026	30/06/2026	Pytest usage
4	Thêm cài đặt phụ thuộc frontend với npm ci và build production React. Cấu hình biến môi trường VITE_API_BASE_URL=/api.	01/07/2026	01/07/2026	npm ci docs
5	Thêm validation cho Docker, triển khai và cấu hình môi trường. Cải thiện workflow logs để dễ dàng truy vết lỗi.	02/07/2026	02/07/2026	Workflow syntax
6	Kiểm thử workflow với các thay đổi từ nhóm, sửa lỗi YAML và đường dẫn, lập tài liệu tích hợp cho commit sau.	03/07/2026	03/07/2026	Troubleshooting

Thành tựu Tuần 3

- Tạo thành công quy trình CI tự động cho kho chứa CloudLibrary.
- Tự động hóa việc kiểm thử API backend và build production frontend.
- Bổ sung việc kiểm tra tính hợp lệ của cấu hình trước khi triển khai.
- Nâng cao khả năng truy vết lỗi tích hợp thông qua nhật ký của workflow.

- Thiết lập quy trình kiểm tra mã nguồn có thể lặp lại cho toàn đội ngũ.

Nhật ký công việc Tuần 4

Thời gian: 06/07/2026 – 10/07/2026

Mục tiêu Tuần 4

- Chuẩn bị định danh, quyền hạn trên AWS và các kho chứa container.
- Kết nối bảo mật GitHub Actions với AWS thông qua OpenID Connect.
- Kiểm thử quá trình build và đẩy Docker image lên Amazon ECR.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Cài đặt và cấu hình AWS CLI, xác thực danh tính tài khoản AWS và chuẩn hóa vùng làm việc là ap-southeast-2.	06/07/2026	06/07/2026	AWS CLI
3	Tạo các private ECR repositories aws-library-backend và aws-library-frontend. Xác định image tags và quy tắc đặt tên.	07/07/2026	07/07/2026	Amazon ECR
4	Cấu hình GitHub OpenID Connect provider và tạo IAM role aws-library-github-deploy với chính sách tin cậy giới hạn theo nhánh.	08/07/2026	08/07/2026	GitHub OIDC
5	Chuẩn bị quyền hạn IAM cho ECR, S3, Elastic Beanstalk, CloudFormation và Auto Scaling. Sử dụng policy simulation.	09/07/2026	09/07/2026	IAM simulator
6	Build, gắn thẻ và đẩy Docker images lên ECR. Kiểm tra image tags, digests và khả năng pull image của AWS resources.	10/07/2026	10/07/2026	Pushing to ECR

Thành tựu Tuần 4

- Cấu hình thành công AWS CLI và vùng làm việc AWS đồng bộ.
- Tạo hai kho chứa Amazon ECR riêng tư.
- Kết nối GitHub Actions với AWS thông qua OIDC mà không cần khóa truy cập cố định.
- Tạo vai trò IAM chuyên dụng và kiểm tra quyền hạn.

- Đẩy thành công các Docker image frontend và backend lên ECR.

Nhật ký công việc Tuần 5

Thời gian: 13/07/2026 – 17/07/2026

Mục tiêu Tuần 5

- Tạo ứng dụng Elastic Beanstalk và môi trường Docker.
- Chuẩn bị gói triển khai và quy trình quản lý phiên bản ứng dụng.
- Hoàn thành lần triển khai AWS đầu tiên của toàn bộ dự án tích hợp.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Tạo ứng dụng Elastic Beanstalk aws-library-system và môi trường Aws-library-system-env. Xem xét EC2 và resources.	13/07/2026	13/07/2026	Beanstalk docs
3	Chuẩn bị production Docker Compose template với ECR image URIs, container dependencies, health checks, ports và env variables.	14/07/2026	14/07/2026	Beanstalk Docker
4	Chuẩn bị Elastic Beanstalk deployment bundle, tải lên S3 bucket và tạo application version.	15/07/2026	15/07/2026	App versions
5	Cấu hình Elastic Beanstalk environment properties cho PostgreSQL, app secrets, logging và region settings.	16/07/2026	16/07/2026	Env properties
6	Thực hiện deployment đầu tiên, kiểm tra public URL và health status, review deployment events và luyện tập rollback.	17/07/2026	17/07/2026	Deploying version

Thành tựu Tuần 5

- Tạo thành công ứng dụng và môi trường CloudLibrary Elastic Beanstalk.
- Chuẩn bị gói triển khai Docker Compose sử dụng ECR images.
- Cấu hình các biến môi trường ứng dụng tách biệt khỏi mã nguồn.
- Tạo và triển khai thành công các phiên bản ứng dụng trên Elastic Beanstalk.

- Hiểu rõ cơ chế sự kiện triển khai và khôi phục phiên bản.

Nhật ký công việc Tuần 6

Thời gian: 20/07/2026 – 24/07/2026

Mục tiêu Tuần 6

- Tự động hóa toàn bộ quá trình triển khai với GitHub Actions.
- Tăng cường bảo mật quyền IAM và kiểm tra xác thực sau triển khai.
- Xác nhận rằng push lên nhánh develop_2.0 có thể tự động cập nhật môi trường AWS.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Mở rộng deployment workflow để build frontend/backend Docker images và đẩy commit-tagged images lên hai ECR repositories.	20/07/2026	20/07/2026	Publishing images
3	Tự động hóa tạo deployment bundle, upload lên S3, tạo Elastic Beanstalk application version và cập nhật môi trường.	21/07/2026	21/07/2026	Create app version
4	Thêm deployment verification loop để so sánh expected version với running version và kiểm tra Status: Ready.	22/07/2026	22/07/2026	Describe envs
5	Khắc phục thiếu IAM permissions cho CloudFormation, S3 và Auto Scaling. Cập nhật GitHub deployment role và EC2 role.	23/07/2026	23/07/2026	EB roles
6	Kiểm thử automated pipeline bằng cách tích hợp team changes và push lên develop_2.0. Xác minh image publication và deployment.	24/07/2026	24/07/2026	Monitoring workflows

Thành tựu Tuần 6

- Hoàn thành pipeline triển khai chính trên GitHub Actions.
- Tự động hóa quá trình xuất bản Docker image và triển khai Elastic Beanstalk.
- Bổ sung việc kiểm tra phiên bản và trạng thái sức khỏe sau khi triển khai.
- Chỉnh sửa các quyền IAM cần thiết cho quá trình triển khai.

- Xác nhận push lên develop_2.0 kích hoạt quá trình triển khai AWS tự động.

Nhật ký công việc Tuần 7

Thời gian: 27/07/2026 – 30/07/2026

Mục tiêu Tuần 7

- Cấu hình CloudWatch monitoring, logs, metrics và alarms.
- Khắc phục các sự cố CI/CD và triển khai Elastic Beanstalk còn tồn đọng.
- Hoàn thành tích hợp cuối cùng, xác thực triển khai và hoàn thiện tài liệu dự án.

Các công việc thực hiện trong tuần

Ngày	Công việc	Ngày bắt đầu	Ngày hoàn thành	Tài liệu tham khảo
2	Cấu hình CloudWatch log collection, metric filters, dashboard widgets và alarms cho CPU, request count, HTTP 5xx và latency.	27/07/2026	27/07/2026	CloudWatch
3	Xác minh app và deployment logs từ Elastic Beanstalk, Docker, Nginx, Flask và Gunicorn. Kiểm thử CloudWatch alarms.	28/07/2026	28/07/2026	Beanstalk logging
4	Giải quyết CI và deployment issues, bao gồm Python import path, thiếu AWS permissions, version mismatch và PostgreSQL credentials.	29/07/2026	29/07/2026	EB troubleshooting
5	Tích hợp frontend/backend changes cuối cùng, cập nhật book-cover assets, push final version lên develop_2.0, verify ECR và EB deployment.	30/07/2026	30/07/2026	Managing EB

Thành tựu Tuần 7

- Hoàn thành CloudWatch monitoring, dashboards, log collection và alarms.
- Chẩn đoán deployment failures qua GitHub Actions, CloudShell, EB events và log bundles.
- Khắc phục backend import path, IAM permissions và PostgreSQL configuration issues.
- Tích hợp và triển khai thành công các sản phẩm bàn giao cuối cùng của nhóm.
- Xác minh toàn bộ luồng từ source-code push đến running AWS environment.

- Hoàn thành tài liệu về kiến trúc, triển khai, troubleshooting và worklog cho báo cáo cuối kỳ.

Tổng kết công việc chung

Trong suốt thời gian thực hiện dự án kéo dài bảy tuần, công việc của tôi tập trung vào hệ thống tích hợp, containerization, AWS deployment, CI/CD automation, monitoring và troubleshooting. Tôi đã tích hợp React frontend, Flask backend và PostgreSQL database sử dụng Docker Compose, sau đó cấu hình GitHub Actions để test, build và deploy ứng dụng tự động.

Frontend và backend Docker images được lưu trữ tại Amazon ECR và deploy lên AWS Elastic Beanstalk. GitHub Actions xác thực với AWS qua OIDC và IAM roles, trong khi Amazon CloudWatch dùng cho log collection, health monitoring và operational metrics.

Một số vấn đề deployment như incorrect import paths, insufficient IAM permissions và inconsistent database credentials đã được nhận diện và giải quyết thông qua GitHub Actions logs, Elastic Beanstalk events, CloudWatch và AWS CloudShell.

Nhìn chung, những đóng góp của tôi đã thiết lập một quy trình deploy có thể lặp lại và dễ dàng truy vết, kết nối frontend và backend thành một ứng dụng cloud-based hoạt động ổn định.

2 Mô tả dự án

2.1 Tổng quan dự án

CloudLibrary là một ứng dụng web quản lý thư viện dựa trên nền tảng đám mây, được phát triển như một phần của Chương trình Thực tập AWS tại Việt Nam. Dự án nhằm mục đích cung cấp một giải pháp kỹ thuật số hoàn chỉnh để quản lý sách, người dùng, hoạt động mượn sách, quy trình trả sách, đặt trước, thông báo và các hoạt động hành chính trong môi trường thư viện.

Hệ thống được thiết kế dưới dạng ứng dụng web full-stack với frontend sử dụng React, backend API sử dụng Flask và cơ sở dữ liệu PostgreSQL. Bên cạnh việc triển khai các tính năng nghiệp vụ cốt lõi, dự án còn tập trung vào việc triển khai trên đám mây, tự động hóa CI/CD, đóng gói container, giám sát và quản lý vận hành bằng các dịch vụ của AWS.

Mục tiêu chính của dự án không chỉ là xây dựng một hệ thống thư viện hoạt động tốt, mà còn để áp dụng các phương pháp DevOps và cloud-native hiện đại. Hệ thống hoàn chỉnh cho phép đội ngũ phát triển cập nhật mã nguồn, đẩy các thay đổi lên GitHub, tự động xây dựng hình ảnh Docker, lưu trữ các hình ảnh trong Amazon ECR, triển khai ứng dụng bằng AWS Elastic Beanstalk và giám sát hệ thống đang chạy thông qua Amazon CloudWatch.

2.2 Mục tiêu dự án

Dự án được lên kế hoạch với các mục tiêu sau:

- Xây dựng một hệ thống mượn và quản lý sách thư viện dựa trên web hoàn chỉnh.
- Triển khai xác thực và phân quyền dựa trên vai trò cho quản trị viên và người dùng thông thường.
- Cung cấp các tính năng quản lý sách như tạo, đọc, cập nhật, xóa, khôi phục, nhập và xuất dữ liệu sách.
- Cho phép người dùng mượn sách, yêu cầu trả sách, gia hạn bản ghi mượn và đặt trước sách hiện đang hết.
- Cung cấp cho quản trị viên các công cụ để quản lý người dùng, sách, bản ghi mượn, đặt trước, thông báo và các hoạt động hệ thống.
- Đóng gói container cho cả ứng dụng frontend và backend bằng Docker.
- Xây dựng quy trình CI/CD tự động bằng GitHub Actions.
- Lưu trữ hình ảnh Docker của frontend và backend trong Amazon Elastic Container Registry.
- Triển khai ứng dụng web trên AWS Elastic Beanstalk.
- Giám sát nhật ký ứng dụng, lỗi, trạng thái sức khỏe, mức sử dụng CPU, mức sử dụng bộ nhớ và các chỉ số hệ thống bằng Amazon CloudWatch.
- Sử dụng AWS CloudShell để thực hiện nhanh các thao tác dòng lệnh AWS và thiết lập quyền.

2.3 Lập kế hoạch

Dự án được lên kế hoạch như một quy trình phát triển từng bước. Nhóm trước tiên tập trung vào các tính năng cốt lõi của ứng dụng, sau đó chuyển sang xác thực, kiểm soát truy cập dựa trên vai trò, các hoạt động thư viện nâng cao, tự động hóa DevOps và triển khai trên AWS.

2.3.1 Giai đoạn 1: Phát triển ứng dụng cốt lõi

Trong giai đoạn đầu, nhóm đã phát triển các tính năng thư viện cơ bản. Backend được thực hiện bằng Flask và Flask-SQLAlchemy, trong khi frontend được phát triển bằng React và Vite. PostgreSQL được chọn làm cơ sở dữ liệu quan hệ vì tính đáng tin cậy, được sử dụng rộng rãi và phù hợp để lưu trữ dữ liệu có cấu trúc như người dùng, sách, bản ghi mượn, đặt trước và thông báo.

Các tính năng chính trong giai đoạn này bao gồm:

- Hiển thị danh sách sách.

- Xem chi tiết sách.
- Tạo, cập nhật và xóa sách.
- Quản lý tình trạng sẵn có của sách.
- Tạo các mô hình cơ sở dữ liệu ban đầu.
- Xây dựng các điểm cuối API cơ bản để tích hợp frontend.

2.3.2 Giai đoạn 2: Xác thực và Phân quyền

Sau khi hoàn thành các tính năng cốt lõi, nhóm đã triển khai xác thực và phân quyền. JSON Web Token được sử dụng để xác thực người dùng. Backend cung cấp các tính năng đăng nhập, đăng ký, thông tin người dùng hiện tại, đặt lại mật khẩu và kiểm tra quyền dựa trên vai trò.

Hệ thống bao gồm hai vai trò chính:

- **Admin (Quản trị viên):** có quyền quản lý sách, người dùng, bản ghi mượn, đặt trước, báo cáo và các hoạt động hệ thống.
- **User (Người dùng):** có thể duyệt sách, mượn sách, trả sách, yêu cầu gia hạn, đặt trước sách, cập nhật thông tin cá nhân và xem lịch sử mượn sách của bản thân.

Giai đoạn này rất quan trọng vì nó tách biệt các hoạt động quản trị khỏi các hoạt động của người dùng thông thường và cải thiện tính bảo mật của hệ thống.

2.3.3 Giai đoạn 3: Tính năng Mượn, Trả, Đặt trước và Thông báo

Giai đoạn tiếp theo tập trung vào việc triển khai các luồng nghiệp vụ thư viện chính. Người dùng có thể mượn sách có sẵn, cung cấp thông tin khách hàng, yêu cầu trả sách, yêu cầu gia hạn và đặt trước những cuốn sách hiện đang hết. Quản trị viên có thể xác nhận trả sách, xem xét tình trạng sách trả, tính toán tiền phạt, phê duyệt hoặc từ chối yêu cầu gia hạn và quản lý hàng đợi đặt trước.

Hệ thống cũng bao gồm các tính năng thông báo để người dùng nhận được cập nhật về các bản ghi mượn, yêu cầu trả sách, trạng thái đặt trước và các hành động hành chính.

2.3.4 Giai đoạn 4: Đóng gói Container và CI/CD

Sau khi các tính năng của ứng dụng hoàn tất, nhóm đã tiến hành container hóa frontend và backend. Docker được sử dụng để đóng gói các ứng dụng thành các môi trường chạy độc lập và có thể tái tạo.

Quy trình CI/CD được xây dựng bằng GitHub Actions. Khi các nhà phát triển sửa mã nguồn và đẩy các thay đổi lên GitHub, GitHub Actions sẽ tự động xây dựng hình ảnh Docker cho frontend và backend. Các hình ảnh này sau đó được đẩy lên Amazon Elastic Container Registry.

Giai đoạn này cải thiện quy trình phát triển vì việc triển khai không còn cần phải thực hiện thủ công nữa. Thay vào đó, các thay đổi mã nguồn có thể được tự động xây dựng và chuẩn bị sẵn sàng để triển khai.

2.3.5 Giai đoạn 5: Triển khai và Giám sát trên AWS

Trong giai đoạn cuối, ứng dụng đã được triển khai lên AWS. Amazon ECR được sử dụng làm registry lưu trữ hình ảnh container cho cả hình ảnh Docker frontend và backend. AWS Elastic Beanstalk được sử dụng để triển khai và quản lý môi trường ứng dụng web đang chạy.

Amazon CloudWatch được sử dụng để giám sát hệ thống đã triển khai. Nó giúp nhóm quan sát nhật ký ứng dụng, phát hiện lỗi, kiểm tra trạng thái sức khỏe của hệ thống và theo dõi việc sử dụng tài nguyên như CPU, bộ nhớ và các chỉ số vận hành khác. AWS CloudShell được sử dụng làm terminal trên trình duyệt để chạy nhanh các lệnh AWS CLI, đặc biệt là cho việc thiết lập quyền và các tác vụ vận hành.

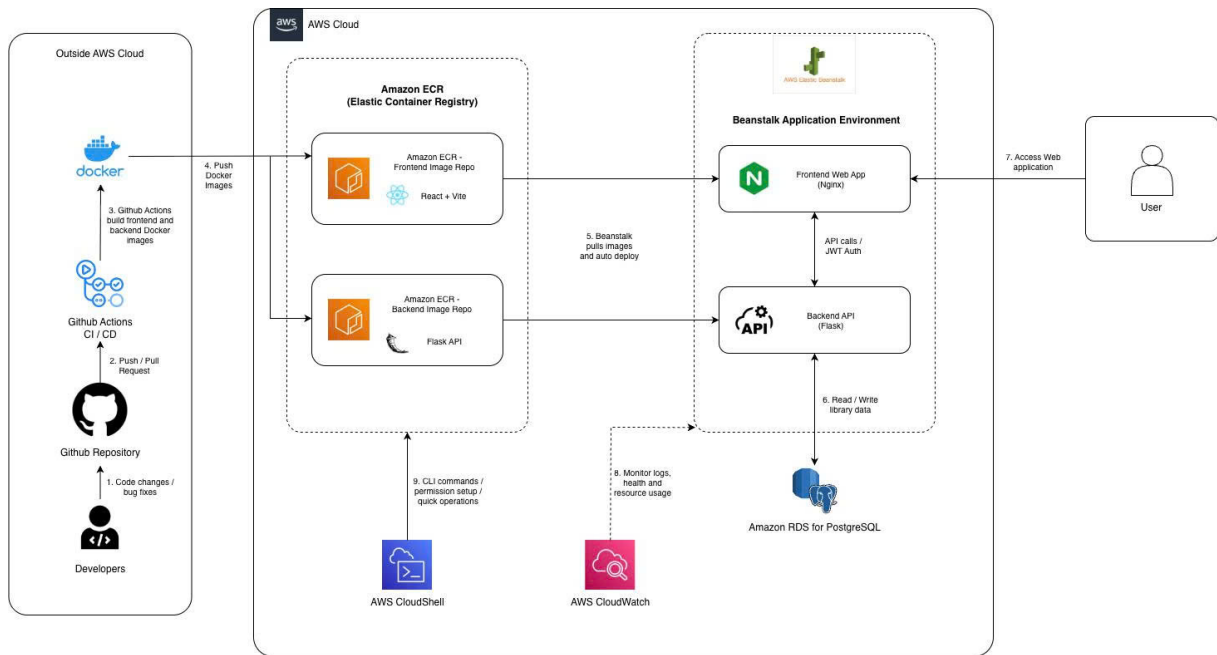
2.4 Ngăn xếp Công nghệ

Tầng	Công nghệ / Dịch vụ
Frontend	React, Vite, Nginx
Backend	Flask, Flask-SQLAlchemy
Xác thực	Xác thực dựa trên JWT
Cơ sở dữ liệu	PostgreSQL
Đóng gói Container	Docker
CI/CD	GitHub Actions
Container Registry	Amazon ECR
Triển khai ứng dụng	AWS Elastic Beanstalk
Giám sát và Ghi nhật ký	Amazon CloudWatch
Vận hành Đám mây	AWS CloudShell

Bảng 1: Ngăn xếp công nghệ của dự án CloudLibrary

2.5 Thiết kế và Kiến trúc

Hệ thống CloudLibrary tuân theo kiến trúc triển khai dựa trên đám mây với CI/CD tự động và giám sát tập trung. Thiết kế tổng thể tách biệt quy trình phát triển, lưu trữ hình ảnh container, môi trường triển khai, tầng cơ sở dữ liệu, luồng truy cập của người dùng và luồng giám sát.



Hình 1: Kiến trúc triển khai CloudLibrary trên AWS

2.6 Phân tích Kiến trúc

Hình 1 minh họa kiến trúc triển khai hoàn chỉnh và quy trình làm việc của dự án CloudLibrary trên AWS. Kiến trúc được tổ chức thành một số phần chính: quản lý mã nguồn, tự động hóa CI/CD, registry hình ảnh container, triển khai ứng dụng, lưu trữ cơ sở dữ liệu, truy cập của người dùng, giám sát và vận hành đám mây.

2.6.1 Phát triển và Quản lý Mã nguồn

Quy trình bắt đầu từ đội ngũ phát triển. Các nhà phát triển sửa lỗi, triển khai tính năng mới và đẩy các thay đổi mã nguồn lên kho lưu trữ GitHub. GitHub được sử dụng làm nền tảng quản lý mã nguồn trung tâm cho cả mã nguồn frontend và backend.

Khi một nhà phát triển đẩy mã nguồn hoặc tạo một pull request, quy trình CI/CD sẽ được kích hoạt. Điều này đảm bảo rằng các thay đổi được kiểm tra và xây dựng tự động trước khi được triển khai. Cách tiếp cận này giúp giảm thiểu lỗi triển khai thủ công và làm cho quá trình phát hành trở nên đồng nhất hơn.

2.6.2 Quy trình CI/CD với GitHub Actions

GitHub Actions chịu trách nhiệm tự động hóa quá trình xây dựng. Sau khi mã nguồn được đẩy lên GitHub, quy trình sẽ xây dựng hình ảnh Docker cho cả hai tầng ứng dụng:

- Hình ảnh Docker Frontend cho ứng dụng web React và Vite.

- Hình ảnh Docker Backend cho dịch vụ API Flask.

Quá trình này đảm bảo rằng frontend và backend được đóng gói trong một môi trường chạy nhất quán. Bằng cách sử dụng hình ảnh Docker, ứng dụng có thể chạy đáng tin cậy hơn trên các môi trường phát triển cục bộ, CI/CD và triển khai trên AWS.

2.6.3 Lưu trữ Hình ảnh Container với Amazon ECR

Amazon Elastic Container Registry được sử dụng để lưu trữ hình ảnh Docker được tạo ra bởi quy trình CI/CD. Dự án sử dụng các kho lưu trữ ECR cho cả hình ảnh frontend và backend.

Mục đích của Amazon ECR trong dự án này là cung cấp một registry lưu trữ hình ảnh container an toàn và được quản lý. Sau khi GitHub Actions xây dựng hình ảnh Docker, nó sẽ đẩy chúng lên ECR. Các hình ảnh này sau đó trở thành các cấu phần triển khai được sử dụng bởi AWS Elastic Beanstalk.

Việc sử dụng ECR giúp tách biệt giai đoạn xây dựng khỏi giai đoạn triển khai. Quy trình CI/CD chỉ cần xây dựng và đẩy hình ảnh, trong khi dịch vụ triển khai sau đó có thể kéo các hình ảnh chính xác từ ECR.

2.6.4 Triển khai Ứng dụng với AWS Elastic Beanstalk

AWS Elastic Beanstalk được sử dụng để triển khai và quản lý ứng dụng web CloudLibrary. Elastic Beanstalk đơn giản hóa việc triển khai bằng cách cung cấp một môi trường ứng dụng được quản lý. Thay vì phải cấu hình thủ công máy chủ, môi trường chạy và các bước triển khai, nhóm có thể triển khai các ứng dụng được đóng gói container thông qua Beanstalk.

Trong kiến trúc này, Elastic Beanstalk kéo hình ảnh Docker frontend và backend từ Amazon ECR và triển khai chúng vào môi trường ứng dụng. Ứng dụng web frontend phục vụ giao diện người dùng, trong khi API Flask backend xử lý logic nghiệp vụ, xác thực, quản lý sách, mượn sách, trả sách, đặt trước, thông báo, báo cáo và các hoạt động hành chính.

Elastic Beanstalk giúp nhóm tập trung nhiều hơn vào việc phát triển ứng dụng và ít bận tâm hơn về quản lý hạ tầng.

2.6.5 Giao tiếp giữa Frontend và Backend

Người dùng truy cập hệ thống CloudLibrary thông qua trình duyệt web. Frontend giao tiếp với backend thông qua các lệnh gọi API. Việc xác thực được xử lý bằng token JWT. Sau khi người dùng đăng nhập, frontend lưu trữ token và gửi nó trong phần header của yêu cầu khi gọi các API backend được bảo vệ.

Backend xác thực token JWT và kiểm tra vai trò của người dùng trước khi cho phép truy cập vào các tài nguyên được bảo vệ. Người dùng quản trị có thể truy cập các tính năng quản lý, trong khi người dùng thông thường có thể truy cập các tính năng mượn sách, đặt trước, hồ sơ và lịch sử cá nhân.

2.6.6 Tầng Cơ sở dữ liệu với PostgreSQL

API backend kết nối với cơ sở dữ liệu PostgreSQL để lưu trữ dữ liệu ứng dụng. Cơ sở dữ liệu chứa các thực thể quan trọng như người dùng, sách, bản ghi mượn, đặt trước, thông báo, nhật ký kiểm toán và token đặt lại mật khẩu.

Backend đọc và ghi dữ liệu vào cơ sở dữ liệu bất cứ khi nào người dùng thực hiện các hành động như đăng nhập, mượn sách, trả sách, đặt trước sách, cập nhật hồ sơ hoặc khi quản trị viên quản lý các hoạt động thư viện.

PostgreSQL được lựa chọn vì hệ thống yêu cầu dữ liệu quan hệ có cấu trúc và mối quan hệ nhất quán giữa người dùng, sách, bản ghi mượn và đặt trước.

2.6.7 Giám sát với Amazon CloudWatch

Amazon CloudWatch được sử dụng để giám sát ứng dụng đã được triển khai. CloudWatch thu thập nhật ký và các chỉ số hệ thống từ môi trường đang chạy. Trong dự án này, CloudWatch được sử dụng để quan sát:

- Nhật ký ứng dụng.
- Lỗi hệ thống.
- Mức sử dụng CPU.
- Mức sử dụng bộ nhớ.
- Trạng thái sức khỏe của ứng dụng.
- Các chỉ số vận hành.
- Cảnh báo và bảng điều khiển giám sát.

CloudWatch giúp nhóm phát hiện các sự cố sau khi triển khai. Ví dụ, nếu backend trả về lỗi, nếu ứng dụng gặp sự cố về sức khỏe, hoặc nếu việc sử dụng CPU và bộ nhớ tăng bất thường, CloudWatch cung cấp khả năng hiển thị để khắc phục sự cố.

Tầng giám sát này rất quan trọng vì việc triển khai đơn thuần là chưa đủ đối với một hệ thống hướng tới môi trường sản xuất. Nhóm cũng cần một phương pháp để quan sát hệ thống sau khi nó đi vào hoạt động.

2.6.8 Vận hành Đám mây với AWS CloudShell

AWS CloudShell được sử dụng như một terminal trên trình duyệt cho các hoạt động trên AWS. Nó cho phép nhóm chạy trực tiếp các lệnh AWS CLI ngay trong bảng điều khiển AWS mà không cần thiết lập môi trường CLI cục bộ.

Trong dự án này, CloudShell hữu ích cho các tác vụ nhanh như thiết lập quyền, kiểm tra tài nguyên AWS, chạy các lệnh liên quan đến triển khai và thực hiện kiểm tra vận hành. Nó giúp giảm thời gian cần thiết để điều hướng thủ công qua các trang dịch vụ AWS khác nhau.

2.7 Quy trình Triển khai

Quy trình triển khai của dự án có thể được tóm tắt như sau:

1. Các nhà phát triển sửa lỗi hoặc triển khai tính năng mới.
2. Mã nguồn được đẩy lên kho lưu trữ GitHub.
3. GitHub Actions tự động xây dựng hình ảnh Docker cho frontend và backend.
4. Hình ảnh Docker được tạo ra được đẩy lên Amazon ECR.
5. AWS Elastic Beanstalk kéo các hình ảnh từ Amazon ECR.
6. Elastic Beanstalk tự động triển khai ứng dụng web.
7. Người dùng truy cập ứng dụng CloudLibrary đã được triển khai thông qua trình duyệt.
8. Backend đọc và ghi dữ liệu vào PostgreSQL.
9. Amazon CloudWatch liên tục giám sát nhật ký, lỗi, trạng thái sức khỏe và mức sử dụng tài nguyên.
10. AWS CloudShell được sử dụng để thực hiện nhanh các lệnh dòng lệnh và thiết lập quyền.

2.8 Thiết kế Bảo mật và Kiểm soát Truy cập

Hệ thống bao gồm xác thực và phân quyền để bảo vệ các tài nguyên ứng dụng. JWT được sử dụng để xác minh danh tính người dùng. Sau khi đăng nhập thành công, người dùng nhận được một token truy cập. Token này phải được đính kèm trong các yêu cầu API được bảo vệ.

Kiểm soát truy cập dựa trên vai trò được áp dụng để phân tách quyền hạn giữa quản trị viên và người dùng thông thường. Người dùng quản trị được phép thực hiện các thao tác nhạy cảm như quản lý sách, người dùng, bản ghi mượn, đặt trước, báo cáo và nhật ký kiểm toán hệ thống. Người dùng thông thường có thể duyệt sách, mượn sách, yêu cầu trả sách, yêu cầu gia hạn, đặt trước sách và quản lý hồ sơ của chính họ.

Thiết kế này giúp giảm thiểu việc truy cập trái phép và đảm bảo rằng người dùng chỉ có thể thực hiện các thao tác phù hợp với vai trò của họ.

2.9 Cân nhắc về Khả năng Mở rộng và Khả năng Bảo trì

Dự án được thiết kế với sự cân nhắc về khả năng mở rộng và bảo trì. Docker giúp môi trường chạy nhất quán hơn. GitHub Actions cải thiện việc tự động hóa bàn giao. Amazon ECR cung cấp một registry lưu trữ hình ảnh container tập trung. Elastic Beanstalk đơn giản hóa việc triển khai và quản lý môi trường. CloudWatch cung cấp khả năng hiển thị vận hành.

Mặc dù phiên bản hiện tại tập trung vào việc triển khai bằng Elastic Beanstalk, kiến trúc này có thể được mở rộng trong tương lai. Các cải tiến có thể bao gồm việc sử dụng Amazon RDS làm cơ sở dữ liệu sản xuất được quản lý, thêm Amazon S3 cho các tài nguyên tĩnh hoặc tệp được tải lên, tích hợp Amazon SES để gửi thông báo qua email, và di chuyển các dịch vụ được đóng gói container sang Amazon ECS hoặc Amazon EKS nếu hệ thống yêu cầu khả năng mở rộng và điều phối nâng cao hơn.

2.10 Kết quả Dự kiến

Các kết quả dự kiến của đề xuất là:

- Một ứng dụng web hoàn chỉnh để quản lý thư viện.
- Hệ thống xác thực và phân quyền dựa trên vai trò hoạt động tốt.
- Quy trình mượn, trả, đặt trước và thông báo hoàn chỉnh.
- Frontend và backend được đóng gói Docker.
- Quy trình CI/CD tự động từ GitHub lên AWS.
- Hình ảnh Docker được lưu trữ an toàn trong Amazon ECR.
- Việc triển khai ứng dụng được quản lý bởi AWS Elastic Beanstalk.
- Hỗ trợ giám sát và khắc phục sự cố thông qua Amazon CloudWatch.
- Tăng tốc các tác vụ vận hành sử dụng AWS CloudShell.

2.11 Kết luận

Đề xuất CloudLibrary tập trung vào việc xây dựng một hệ thống quản lý thư viện dựa trên đám mây thực tế đồng thời áp dụng việc triển khai trên AWS và các thực hành DevOps. Kiến trúc này kết hợp việc phát triển ứng dụng, đóng gói container, tự động hóa CI/CD, triển khai trên AWS và giám sát thành một quy trình hoàn chỉnh.

Thông qua thiết kế này, dự án minh họa cách một ứng dụng web full-stack có thể được phát triển cục bộ, đóng gói bằng Docker, tích hợp với GitHub Actions, lưu trữ trong Amazon ECR, triển khai bằng AWS Elastic Beanstalk và giám sát bởi Amazon CloudWatch. Điều này cung

cấp cả một ứng dụng hoạt động tốt cho các hoạt động thư viện và trải nghiệm học tập thực tế về phân phối phần mềm cloud-native trên AWS.

3 Các bài Blog

Blog 1 - XÂY DỰNG CƠ CHẾ QUẢN LÝ TOKEN VÀ TỰ ĐỘNG LÀM MỚI PHIÊN

Bài viết này giới thiệu mô hình xác thực và xử lý phiên làm việc trong ứng dụng React & Fetch API, cho phép lưu trữ token linh hoạt giữa localStorage hoặc sessionStorage tùy theo sở thích của người dùng. Hệ thống tích hợp một tầng request tập trung, tự động đính kèm tiêu đề Authorization và xử lý lỗi tự động khi gặp mã trạng thái HTTP 401 Unauthorized, từ đó phát đi một sự kiện toàn cục để chuyển hướng an toàn mà không làm sập ứng dụng.

Blog 2 - TỐI ƯU HÓA TÌM KIẾM ĐA TIÊU CHÍ VÀ LỌC DỮ LIỆU

Bài viết tập trung phân tích các kỹ thuật tối ưu hóa hiệu năng phía client cho các danh sách lớn bằng cách sử dụng hook useMemo kết hợp với việc kết hợp chuỗi tìm kiếm đa trường thông minh (tiêu đề, tác giả, thể loại, ISBN, vị trí). Ngoài ra, hệ thống áp dụng cơ chế lọc trạng thái đa điều kiện kết hợp cùng các thuật toán sắp xếp linh hoạt hỗ trợ chuẩn tiếng Việt thông qua localeCompare, nâng cao trải nghiệm tra cứu mượn mà cho người dùng.

Blog 3 - XÂY DỰNG QUY TRÌNH NGHIỆP VỤ MƯỢN - TRẢ SÁCH VÀ KIỂM TRA ĐIỀU KIỆN

Bài viết trình bày mô hình thiết kế cho quy trình nghiệp vụ thư viện số sử dụng biểu mẫu modal nhiều bước, giúp kiểm tra chặt chẽ các điều kiện trước khi xác nhận giao dịch như hạn trả, tiền cọc và thông tin độc giả. Đồng thời, kiến trúc giao diện người dùng dựa trên phân quyền linh hoạt giúp phân định rõ ràng quyền hạn giữa Quản trị viên (phê duyệt gia hạn, nhận sách trả, xử lý tiền phạt) và Độc giả (gửi yêu cầu mượn, trả hoặc xin gia hạn) một cách an toàn và minh bạch.

4 Các sự kiện đã tham gia

Sự kiện 1

- **Tên sự kiện:** First Cloud AI Journey – Chuỗi hội thảo kỹ thuật (Bảo mật ứng dụng web, Lộ trình chứng chỉ AWS, và Giám sát SLA)
- **Thời gian:** 09:00, ngày 11 tháng 07 năm 2026

- **Địa điểm:** Tầng 26, Tòa nhà Bitexco Financial Tower, Thành phố Hồ Chí Minh
- **Vai trò:** Người tham dự
- **Mô tả ngắn gọn về nội dung và các hoạt động chính của sự kiện:**
 - **Nội dung 1 (Bảo mật ứng dụng web):** Khám phá "Frontier Agent" được hỗ trợ bởi Amazon Bedrock, một tác nhân bảo mật tự động có khả năng lên kế hoạch và thực thi các tác vụ bảo mật trong toàn bộ vòng đời—bao gồm Đánh giá thiết kế, Bảo mật mã nguồn, và Kiểm thử xâm nhập chủ động.
 - **Nội dung 2 (Lộ trình chứng chỉ AWS):** Cung cấp lộ trình chiến lược để chinh phục kỳ thi AWS Certified Cloud Practitioner (CLF-C02), phân tích chi tiết bốn miền cốt lõi, cấu trúc bài thi, hình thức thi (Pearson VUE hoặc trực tuyến), cùng các mẹo và thủ thuật thực tế.
 - **Nội dung 3 (SLA và Giám sát):** Thảo luận về tầm quan trọng của Thỏa thuận Cấp độ Dịch vụ (SLA) trong quản lý rủi ro, khoảng cách giữa "hạ tầng khỏe mạnh" và "trải nghiệm người dùng tốt", cũng như cách thiết lập các luồng giám sát và cảnh báo hiệu quả sử dụng CloudWatch và SNS.
- **Kết quả hoặc giá trị thu được (bài học kinh nghiệm, kỹ năng mới, đóng góp cho nhóm/dự án):**
 - Thu thập kiến thức nâng cao về việc tận dụng AI và các tác nhân tự động để bảo mật các ứng dụng web hiện đại và tự động hóa quá trình quét lỗ hổng.
 - Tiếp thu lộ trình rõ ràng và các phương pháp ôn tập chiến lược để đạt được các chứng chỉ nền tảng của AWS.
 - Hiểu rõ tư duy quan trọng rằng các chỉ số máy chủ tốt không đảm bảo trải nghiệm người dùng tốt, đồng thời học cách giám sát đúng đắn các chỉ số kinh doanh và SLA để ngăn chặn các vấn đề ảnh hưởng trực tiếp đến khách hàng.

Sự kiện 2

- **Tên sự kiện:** Xây dựng Cơ sở Tri thức Đồ thị với AWS Neptune cho GraphRAG
- **Thời gian:** 09:00, ngày 06 tháng 06 năm 2026
- **Địa điểm:** Tầng 26, Tòa nhà Bitexco Financial Tower, Thành phố Hồ Chí Minh
- **Vai trò:** Người tham dự
- **Mô tả ngắn gọn về nội dung và các hoạt động chính của sự kiện:**

- **Nội dung 1 (Tổng quan về GraphRAG & Hạn chế của Vector RAG truyền thống):** Khám phá các hạn chế của Vector RAG khi thiếu bối cảnh dữ liệu toàn cục, đồng thời xem xét giải pháp được cung cấp bởi GraphRAG sử dụng Đồ thị Tri thức để giúp LLM hiểu rõ các mối quan hệ giữa các thực thể.
 - **Nội dung 2 (AWS Neptune - Trái tim của Cơ sở Tri thức Đồ thị):** Tìm hiểu về kiến trúc của các cơ sở dữ liệu đồ thị được quản lý toàn diện trên AWS hỗ trợ Đồ thị Thuộc tính (Apache TinkerPop Gremlin), RDF/SPARQL, và Neptune ML với các mô hình GNN tích hợp sẵn.
 - **Nội dung 3 (Xây dựng Pipeline Tri thức cho GraphRAG trên AWS):** Đề cập đến quy trình đầu cuối từ việc thu thập và trích xuất dữ liệu, lưu trữ cấu trúc đồ thị kết hợp với tìm kiếm vector trong Neptune, cho đến khâu truy xuất và sinh nội dung để đưa ra câu trả lời chính xác.
- **Kết quả hoặc giá trị thu được (bài học kinh nghiệm, kỹ năng mới, đóng góp cho nhóm/dự án):**
 - Nắm vững kiến thức chuyên sâu về cơ sở dữ liệu đồ thị AWS Neptune và cách áp dụng chúng vào các hệ thống Tạo sinh AI (Generative AI) nâng cao.
 - Hiểu cách triển khai các mô hình tìm kiếm kết hợp (hybrid search) giữa tìm kiếm vector và đồ thị tri thức để tối ưu hóa độ chính xác của RAG.
 - Có được các kỹ năng thực tế trong việc xây dựng các luồng dữ liệu (data pipelines) tích hợp LLM (LangChain/LlamaIndex) với cơ sở dữ liệu đồ thị.

5 Hội thảo: Triển khai CloudLibrary lên AWS

5.1 Tổng quan

Phần workshop này mô tả toàn bộ quy trình được sử dụng để triển khai **CloudLibrary – Hệ thống Mượn Thư viện** lên Amazon Web Services. Mục đích chính là xây dựng một luồng triển khai tự động cho phép một phiên bản ứng dụng mới được kiểm thử, đóng gói container và triển khai bất cứ khi nào mã nguồn được đẩy (push) lên GitHub.

CloudLibrary được triển khai dưới dạng một ứng dụng web full-stack được container hóa. Các thành phần kỹ thuật chính của hệ thống bao gồm:

- **Frontend:** ReactJS và Nginx.
- **Backend:** Python Flask và Gunicorn.
- **Cơ sở dữ liệu (Database):** PostgreSQL.
- **Nền tảng Container:** Docker và Docker Compose.

- **Kho lưu trữ mã nguồn:** GitHub.
- **Nền tảng CI/CD:** GitHub Actions.
- **Container registry:** Amazon Elastic Container Registry (ECR).
- **Nền tảng triển khai:** AWS Elastic Beanstalk.
- **Nền tảng giám sát:** Amazon CloudWatch.

Quy trình triển khai được kích hoạt khi một commit mới được đẩy lên nhánh `develop_2.0`. GitHub Actions trước tiên sẽ xác thực mã nguồn, chạy các bài kiểm thử (test) backend, và build frontend. Sau đó, nó sẽ build các Docker image, đẩy chúng lên Amazon ECR, tạo một phiên bản ứng dụng Elastic Beanstalk mới, và triển khai phiên bản mới này lên môi trường đang chạy.

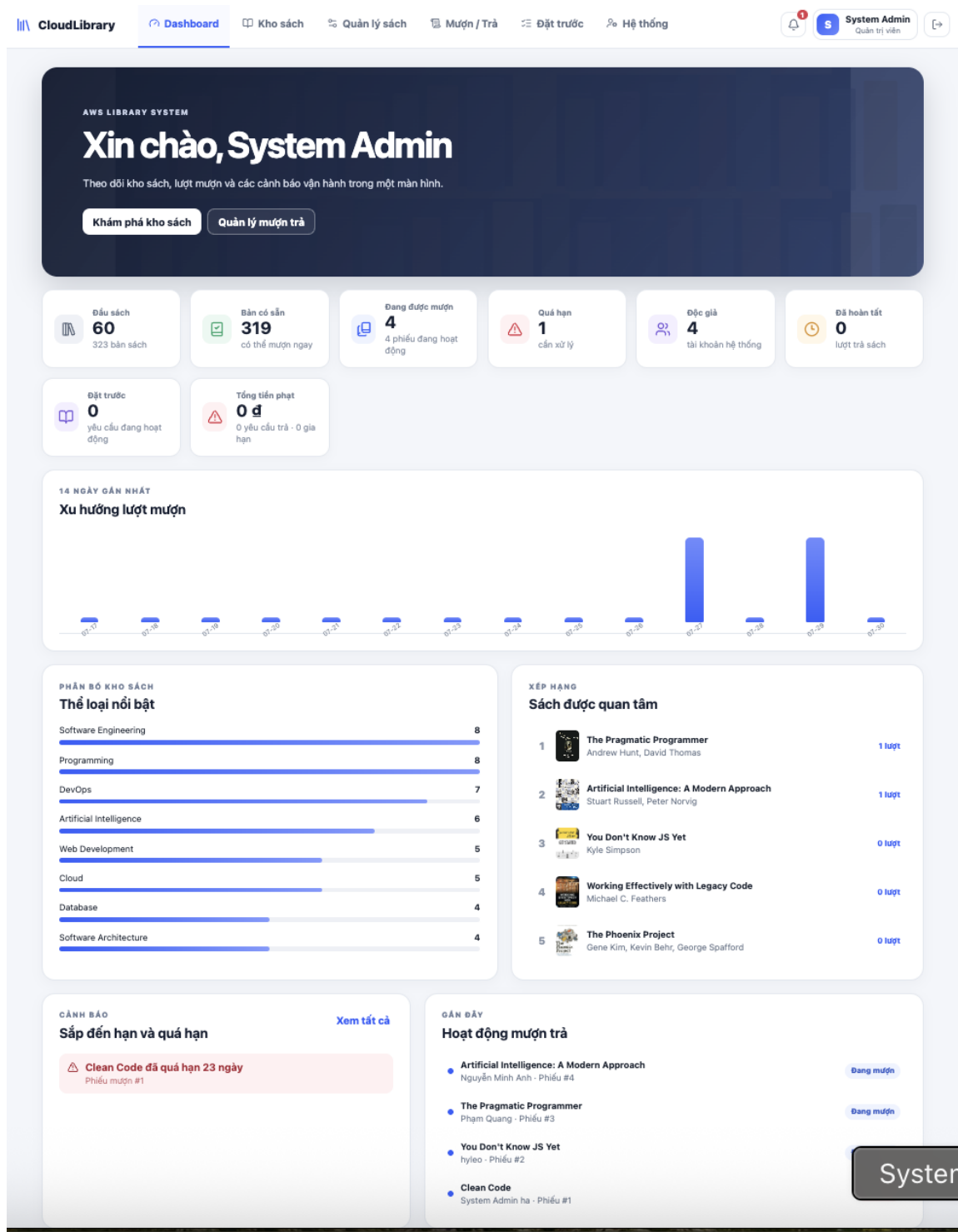
5.2 Điều kiện tiên quyết

Trước khi triển khai dự án, cần chuẩn bị các công cụ và tài nguyên sau:

- Một tài khoản AWS có quyền sử dụng IAM, Amazon ECR, Elastic Beanstalk, Amazon S3, EC2, và CloudWatch.
- Một kho lưu trữ (repository) GitHub chứa mã nguồn CloudLibrary.
- Git đã được cài đặt trên máy tính phát triển.
- Docker Desktop và Docker Compose để kiểm thử cục bộ (local).
- Python 3 và pip để chạy Flask backend và các bài kiểm thử API.
- Node.js và npm để build React frontend.
- AWS CLI để cấu hình các tài nguyên AWS và khắc phục sự cố.
- Một IAM role cho GitHub Actions được cấu hình thông qua OpenID Connect.

Dự án sử dụng các tài nguyên AWS sau:

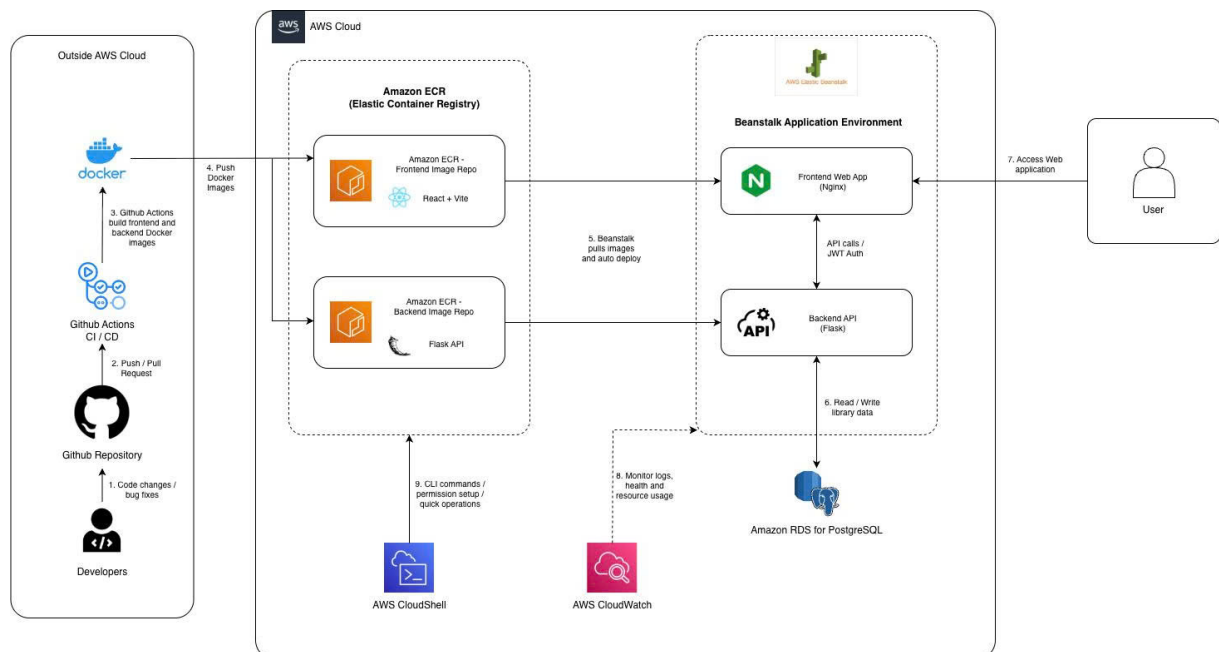
- AWS Region: `ap-southeast-2`.
- Backend ECR repository: `aws-library-backend`.
- Frontend ECR repository: `aws-library-frontend`.
- Elastic Beanstalk application: `aws-library-system`.
- Elastic Beanstalk environment: `Aws-library-system-env`.
- GitHub Actions IAM role: `aws-library-github-deploy`.
- Elastic Beanstalk EC2 instance role: `aws-elasticbeanstalk-ec2-role`.



Hình 2: Giao diện chính của hệ thống CloudLibrary

5.3 Kiến trúc triển khai

Kiến trúc triển khai của CloudLibrary được minh họa trong Hình 3.



Hình 3: Kiến trúc triển khai của CloudLibrary trên AWS

Vai trò của các dịch vụ chính như sau:

- **Git****Hub** lưu trữ mã nguồn và kích hoạt luồng triển khai khi có một commit mới được đẩy lên.
- **Git****Hub** **Actions** tự động hóa quá trình kiểm thử, build Docker image, đẩy image, và triển khai lên Elastic Beanstalk.
- **Amazon** **ECR** lưu trữ các Docker image riêng tư (private) cho frontend và backend.
- **Elastic** **Beanstalk** quản lý EC2 instance, các phiên bản ứng dụng, quá trình triển khai Docker, tình trạng môi trường (environment health), và cung cấp URL công khai.
- **Amazon** **S3** lưu trữ các gói triển khai (deployment bundles) được sử dụng bởi Elastic Beanstalk. S3 cũng có thể được sử dụng sau này để lưu trữ hình ảnh tải lên bởi người dùng hoặc quản trị viên.
- **Amazon** **CloudWatch** thu thập logs của ứng dụng, các chỉ số (metrics) hạ tầng, và thông tin triển khai.
- **AWS** **CloudShell** được sử dụng để thực thi các lệnh AWS CLI, cấu hình quyền IAM, và điều tra các lỗi triển khai.

5.4 Quy trình triển khai

5.4.1 Bước 1: Kiểm tra Cấu trúc Dự án

Trước khi triển khai, cấu trúc dự án được kiểm tra để đảm bảo rằng tất cả các tệp cần thiết đều có sẵn.

Các thư mục và tệp tin quan trọng bao gồm:

```
.github/workflows/  
.platform/hooks/  
aws/  
backend/  
cloudwatch/  
deploy/  
frontend/  
docker-compose.yml  
.env.example
```

Thư mục backend chứa Flask API, trong khi thư mục frontend chứa ứng dụng React và cấu hình Nginx.

Thư mục `.github/workflows` chứa các workflow của GitHub Actions. Thư mục `deploy` chứa template Docker Compose được sử dụng để tạo gói triển khai Elastic Beanstalk.

Tuyệt đối không được commit các tệp nhạy cảm như `.env`, mật khẩu, AWS access keys, và session tokens lên kho lưu trữ GitHub.

5.4.2 Bước 2: Kiểm thử Ứng dụng Cục bộ (Local)

Toàn bộ ứng dụng được kiểm thử cục bộ bằng Docker Compose trước khi được triển khai lên AWS.

Một tệp môi trường (environment file) cục bộ được tạo từ cấu hình mẫu:

```
cp .env.example .env
```

Ứng dụng sau đó được build và khởi chạy:

```
docker compose up --build -d
```

Trạng thái của tất cả các container được kiểm tra bằng lệnh:

```
docker compose ps
```

Các dịch vụ dự kiến (expected services) bao gồm:

db
backend
frontend

Dịch vụ PostgreSQL phải chuyển sang trạng thái "healthy" (khỏe mạnh) trước khi backend khởi động. Backend cũng phải vượt qua kiểm tra sức khỏe (health check) trước khi frontend có thể truy cập được.

Endpoint kiểm tra sức khỏe của backend được test bằng lệnh:

```
curl http://localhost/health
```

Một phản hồi (response) thành công sẽ tương tự như sau:

```
{  
  "service": "library-flask-api",  
  "status": "ok"  
}
```

Trang web cục bộ sau đó có thể được truy cập tại:

`http://localhost`

5.4.3 Bước 3: Kiểm chứng Backend

Các bài kiểm thử backend được thực thi trước khi tạo bản triển khai.

Môi trường ảo Python (virtual environment) được tạo:

```
cd backend  
python3 -m venv .venv  
source .venv/bin/activate
```

Cài đặt các gói phụ thuộc (packages) cần thiết:

```
python -m pip install --upgrade pip  
python -m pip install -r requirements-dev.txt
```

Sau đó chạy các bài test API backend:

```
PYTHONPATH=. python -m pytest -q
```

Các bài test này xác minh những hoạt động quan trọng của backend như xác thực (authentication), phân quyền dựa trên vai trò (authorization), quản lý sách, mượn, trả, gia hạn, đặt trước sách, và các endpoint health-check.

Nếu có bất kỳ bài test bắt buộc nào thất bại, workflow triển khai sẽ bị dừng lại.

5.4.4 Bước 4: Build Frontend

Ứng dụng React được build ở chế độ production trước khi được đóng gói vào Docker image.

Sử dụng các lệnh sau:

```
cd frontend
npm ci
VITE_API_BASE_URL=/api npm run build
```

Lệnh này tạo ra các tệp frontend đã được tối ưu hóa trong thư mục dist.

Giá trị /api được sử dụng vì Nginx sẽ chuyển tiếp (forward) các request API từ frontend tới Flask backend bên trong môi trường Docker Compose.

5.4.5 Bước 5: Tạo các kho lưu trữ (repository) Amazon ECR

Hai kho lưu trữ Amazon ECR riêng tư (private) được sử dụng:

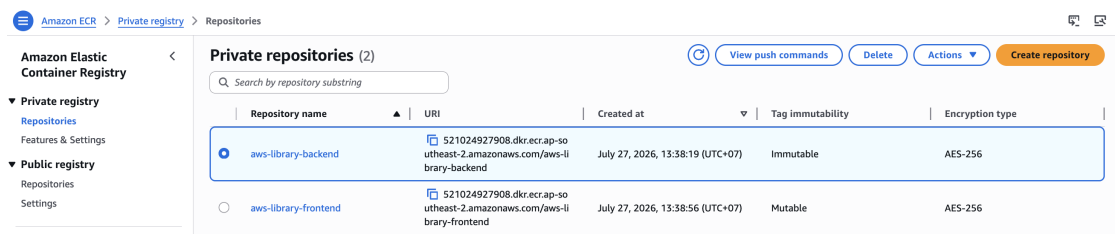
- aws-library-backend lưu trữ các backend image.
- aws-library-frontend lưu trữ các frontend image.

Các kho lưu trữ này có thể được tạo bằng AWS Console hoặc AWS CLI:

```
aws ecr create-repository \
  --repository-name aws-library-backend \
  --region ap-southeast-2
```

```
aws ecr create-repository \
  --repository-name aws-library-frontend \
  --region ap-southeast-2
```

Mỗi image được gán một version tag (thẻ phiên bản) dựa trên GitHub commit. Điều này giúp mọi Docker image đều có thể truy xuất nguồn gốc (traceable) từ mã nguồn đã tạo ra nó.



Hình 4: Các kho lưu trữ Amazon ECR cho các Docker image của CloudLibrary

5.4.6 Bước 6: Cấu hình Xác thực GitHub với AWS

Dự án sử dụng GitHub OpenID Connect thay vì lưu trữ vĩnh viễn các AWS access keys.

GitHub Actions sẽ "assume" (đảm nhận) IAM role sau:

```
aws-library-github-deploy
```

Chính sách tin cậy (trust policy) cho phép kho lưu trữ GitHub và nhánh develop_2.0 yêu cầu thông tin xác thực AWS tạm thời.

IAM role cung cấp các quyền cần thiết để:

- Đăng nhập vào Amazon ECR.
- Đẩy (Push) các Docker images.
- Tải gói triển khai lên Amazon S3.
- Tạo các phiên bản ứng dụng Elastic Beanstalk.
- Cập nhật môi trường Elastic Beanstalk.
- Đọc các sự kiện (events) triển khai và thông tin tình trạng môi trường (environment health).

Việc sử dụng OpenID Connect giúp tránh phải lưu trữ thông tin xác thực AWS dài hạn trong GitHub Secrets và tăng cường bảo mật cho quá trình triển khai.

5.4.7 Bước 7: Cấu hình Elastic Beanstalk

Một ứng dụng Elastic Beanstalk có tên:

```
aws-library-system
```

được tạo ra.

Sau đó, một môi trường dựa trên Docker có tên:

```
Aws-library-system-env
```

được khởi tạo bằng platform Docker trên hệ điều hành Amazon Linux.

Môi trường này chạy ứng dụng bằng Docker Compose và bao gồm ba dịch vụ:

- db: Cơ sở dữ liệu PostgreSQL.
- backend: API service dùng Flask và Gunicorn.
- frontend: Ứng dụng React được phân phối bởi Nginx.

Các image của backend và frontend được kéo (pull) từ Amazon ECR.

Các thuộc tính môi trường (environment properties) sau được cấu hình trong Elastic Beanstalk:

```
POSTGRES_USER
POSTGRES_PASSWORD
POSTGRES_DB
SECRET_KEY
AWS_REGION
LOG_LEVEL
```

URL của cơ sở dữ liệu được tạo ra từ các biến môi trường PostgreSQL:

```
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}
@db:5432/${POSTGRES_DB}
```

Phương pháp này đảm bảo rằng backend và container PostgreSQL sử dụng cùng một bộ thông tin xác thực (credentials).

Mật khẩu và các giá trị bí mật được cấu hình trực tiếp trong Elastic Beanstalk và không được lưu trữ trong kho mã nguồn GitHub.

5.4.8 Bước 8: Cấu hình EC2 Instance Role

EC2 instance chạy Elastic Beanstalk sử dụng role:

```
aws-elasticbeanstalk-ec2-role
```

Role này yêu cầu các quyền để:

- Pull Docker image từ Amazon ECR.
- Gửi logs và số liệu (metrics) đến Amazon CloudWatch.
- Giao tiếp với AWS Systems Manager khi tính năng quản lý từ xa được sử dụng.
- Truy cập các đối tượng Amazon S3 cần thiết cho quá trình triển khai.

Các chính sách (policies) do AWS quản lý sau đây có thể được đính kèm khi cần:

```
CloudWatchAgentServerPolicy
AmazonSSMManagedInstanceCore
```

Nếu các hình ảnh do người dùng tải lên được lưu trữ trên Amazon S3, instance role cũng cần được cấp một số quyền giới hạn như `s3:GetObject`, `s3:PutObject`, và `s3:DeleteObject` đối với bucket hình ảnh của ứng dụng.

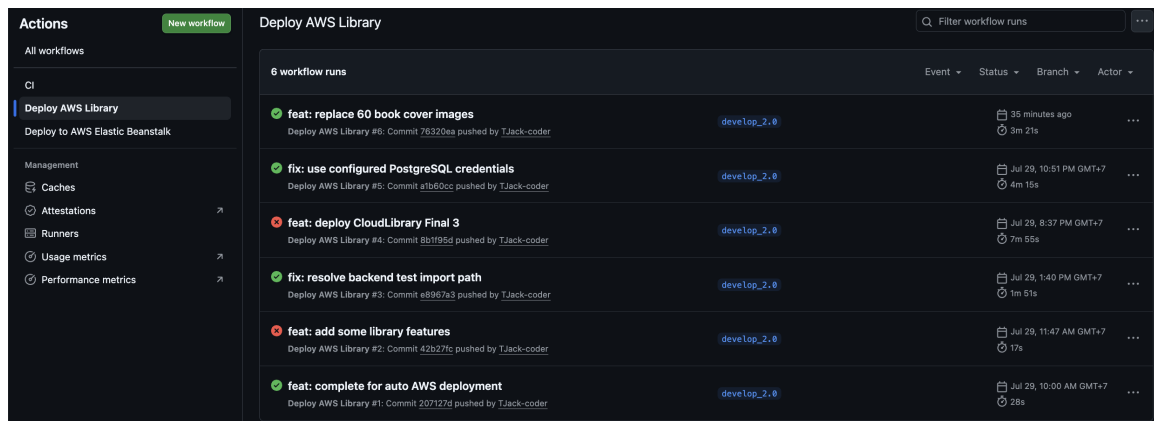
5.4.9 Bước 9: Cấu hình Workflow GitHub Actions

Workflow triển khai được cấu hình để chạy khi mã nguồn mới được push lên nhánh:

`develop_2.0`

Workflow này thực hiện các thao tác sau:

1. Check out mã nguồn.
2. Cài đặt các phần phụ thuộc (dependencies) của Python.
3. Chạy test API backend.
4. Cài đặt các phần phụ thuộc của Node.js.
5. Build React frontend.
6. Yêu cầu thông tin xác thực AWS tạm thời thông qua OpenID Connect.
7. Đăng nhập vào Amazon ECR.
8. Build backend Docker image.
9. Build frontend Docker image.
10. Đẩy (Push) cả hai Docker image lên Amazon ECR.
11. Tạo gói triển khai Elastic Beanstalk.
12. Tải gói triển khai lên Amazon S3.
13. Tạo một phiên bản ứng dụng Elastic Beanstalk mới.
14. Cập nhật môi trường Elastic Beanstalk.
15. Chờ quá trình triển khai hoàn tất.
16. Kiểm tra tính trạng sức khỏe môi trường (environment health) và phiên bản ứng dụng.



Hình 5: Workflow GitHub Actions được sử dụng để xác thực và triển khai CloudLibrary

5.4.10 Bước 10: Push Mã nguồn lên GitHub

Sau khi ứng dụng đã vượt qua khâu kiểm thử cục bộ, mã nguồn sẽ được đẩy (push) lên nhánh triển khai.

Thực thi các lệnh sau trong local Git repository:

```
git switch develop_2.0
git add -A
git commit -m "feat: deploy CloudLibrary to AWS"
git push origin develop_2.0
```

Sự kiện push này sẽ tự động khởi chạy workflow của GitHub Actions.

Sau khi pipeline đã được cấu hình, không cần phải tải thủ công (manual) Docker image lên hay triển khai thủ công trên Elastic Beanstalk nữa.

5.4.11 Bước 11: Build và Push Docker Images

GitHub Actions build các image của backend và frontend một cách riêng biệt.

Backend image chứa:

- Môi trường thực thi (runtime) Python.
- Mã nguồn Flask.
- Gunicorn server.
- Các dependencies của backend.

Frontend image chứa:

- Các tệp React đã được biên dịch.
- Web server Nginx.

- Cấu hình Reverse-proxy cho /api.
- Các tài nguyên tĩnh (static assets) và hình ảnh bìa sách.

Các image sẽ được gắn thẻ (tagged) và đẩy lên hai kho lưu trữ ECR riêng biệt.

Workflow sử dụng định danh commit (commit identifier) trong image tag, do đó các Docker image đã triển khai có thể được đối chiếu dễ dàng với một GitHub commit cụ thể.

5.4.12 Bước 12: Tạo Gói Triển khai (Deployment Bundle)

Sau khi các Docker image đã được đẩy lên ECR, GitHub Actions sẽ tạo một gói triển khai Elastic Beanstalk.

Gói triển khai này chứa một tệp Docker Compose đã được render cùng với các địa chỉ image ECR tương ứng.

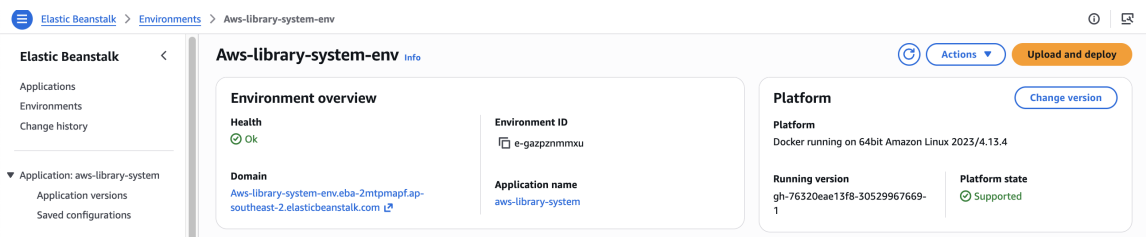
Gói này được nén thành tệp ZIP và tải lên bucket S3 được Elastic Beanstalk sử dụng.

GitHub Actions sau đó tạo ra một phiên bản ứng dụng với định dạng tên tương tự như:

gh-<commit-sha>-<workflow-run-id>-<attempt>

Định dạng tên này giúp ta dễ dàng nhận diện:

- GitHub commit được sử dụng cho đợt triển khai.
- ID của workflow run đã tạo ra phiên bản này.
- Số thứ tự lần thử (attempt) của workflow.



Hình 6: Phiên bản ứng dụng Elastic Beanstalk

5.4.13 Bước 13: Triển khai Ứng dụng

GitHub Actions cập nhật môi trường Elastic Beanstalk để sử dụng phiên bản ứng dụng mới vừa được tạo.

Trong quá trình triển khai, Elastic Beanstalk thực hiện các thao tác sau:

1. Tải xuống gói triển khai từ Amazon S3.

2. Đọc cấu hình Docker Compose.
3. Xác thực (Authenticate) với Amazon ECR.
4. Kéo (Pull) các Docker image của backend và frontend.
5. Dừng các container của phiên bản ứng dụng cũ.
6. Khởi động container PostgreSQL.
7. Chờ PostgreSQL vượt qua health check (kiểm tra sức khỏe).
8. Khởi động container backend.
9. Chờ backend vượt qua health check.
10. Khởi động container Nginx frontend.
11. Chạy các platform deployment hooks (nếu có).
12. Báo cáo kết quả triển khai lại cho Elastic Beanstalk.

Thứ tự triển khai rất quan trọng vì backend phụ thuộc vào PostgreSQL, trong khi frontend lại phụ thuộc vào backend.

5.4.14 Bước 14: Xác minh Triển khai

Sau khi Elastic Beanstalk nhận được phiên bản ứng dụng mới, GitHub Actions sẽ liên tục kiểm tra tình trạng của môi trường.

Việc triển khai được coi là thành công khi:

- Trạng thái (Status) của môi trường là Ready.
- Sức khỏe (Health) của môi trường là Ok.
- Phiên bản đang chạy (Running version) khớp với phiên bản mới được tạo.
- Public health endpoint trả về HTTP status code là 200.

Trạng thái mong đợi (expected status) là:

Status: Ready

Health: Ok

Running version: gh-<new-version>

Website sau đó có thể được truy cập thông qua URL công khai của môi trường Elastic Beanstalk.

5.4.15 Bước 15: Giám sát Ứng dụng

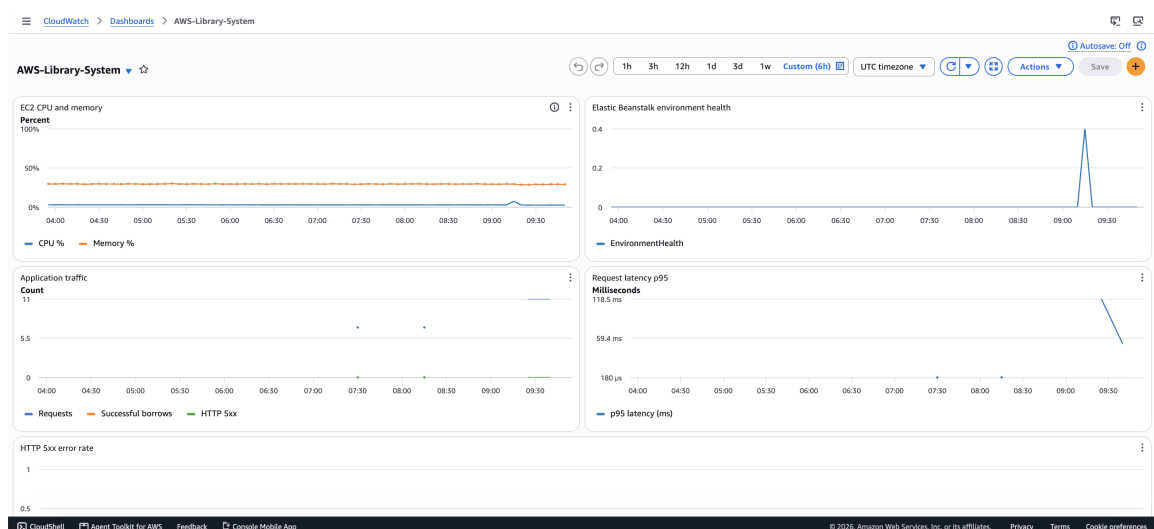
Amazon CloudWatch được sử dụng để thu thập log và giám sát môi trường đang vận hành.

Các thông tin được giám sát bao gồm:

- Tỷ lệ sử dụng CPU của EC2.
- Số lượng HTTP request.
- Độ trễ (Latency) phản hồi của backend.
- Các mã lỗi HTTP 4xx và 5xx.
- Log của Flask và Gunicorn.
- Log truy cập (access logs) và log lỗi (error logs) của Nginx.
- Log của Docker và Docker Compose.
- Các sự kiện (events) triển khai của Elastic Beanstalk.

Cảnh báo (CloudWatch alarms) có thể được thiết lập cho các điều kiện như:

- Mức sử dụng CPU cao.
- Độ trễ backend cao.
- Số lượng phản hồi HTTP 5xx quá lớn.
- Tình trạng (Health) của Elastic Beanstalk chuyển sang Degraded (Xuống cấp) hoặc Severe (Nghiêm trọng).



Hình 7: Giám sát môi trường CloudLibrary bằng Amazon CloudWatch

5.4.16 Bước 16: Khắc phục sự cố (Troubleshoot) khi Triển khai Thất bại

Khi quá trình triển khai thất bại, GitHub Actions sẽ báo lỗi trong deployment job. Elastic Beanstalk cũng có thể chuyển health của môi trường sang trạng thái Degraded, Severe, hoặc Red.

Các nguồn sau đây được sử dụng để xác định vấn đề:

- Log của các job trên GitHub Actions.
- Sự kiện của Elastic Beanstalk (Elastic Beanstalk Events).
- Deployment Logs của Elastic Beanstalk.
- Chế độ Enhanced Health của Elastic Beanstalk.
- Tập `/var/log/eb-engine.log`.
- Log của các Docker container.
- CloudWatch Logs.

Bạn có thể yêu cầu gói log (log bundle) của Elastic Beanstalk thông qua AWS CLI:

```
aws elasticbeanstalk request-environment-info \
--environment-name Aws-library-system-env \
--info-type bundle \
--region ap-southeast-2
```

Trong quá trình triển khai dự án, một lỗi đã xảy ra do kết nối cơ sở dữ liệu backend sử dụng hard-code mật khẩu PostgreSQL, trong khi container PostgreSQL lại sử dụng mật khẩu được cấu hình trong Elastic Beanstalk.

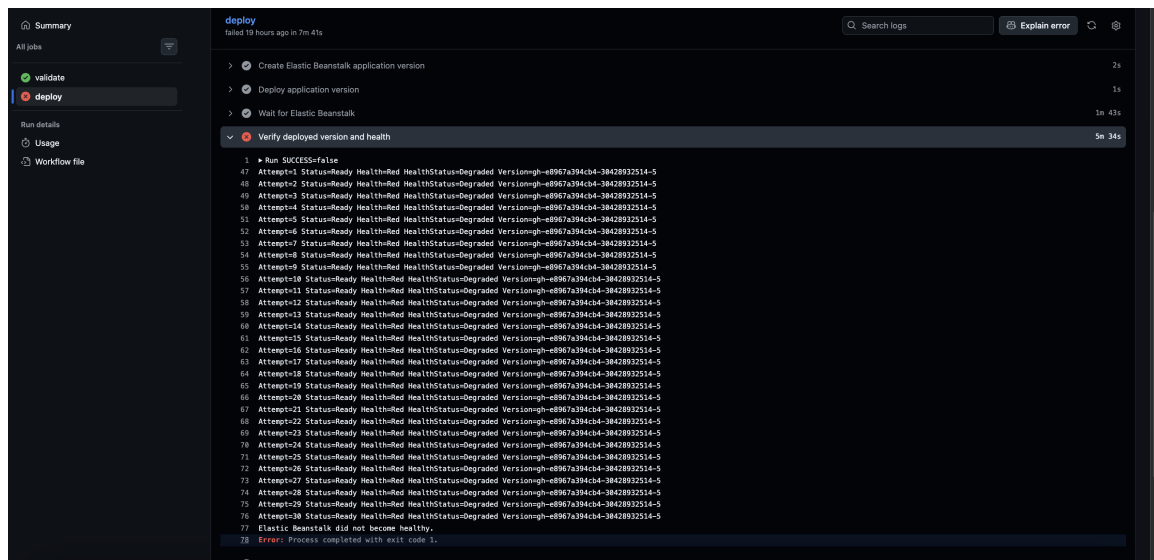
Cấu hình sai này đã gây ra chuỗi sự cố sau:

Không khớp thông tin đăng nhập database → lỗi kết nối backend → lỗi health-check của backend → frontend không khởi động → triển khai Elastic Beanstalk thất bại

Vấn đề này đã được giải quyết bằng cách cấu trúc lại URL cơ sở dữ liệu dựa trên:

- POSTGRES_USER.
- POSTGRES_PASSWORD.
- POSTGRES_DB.

Sau khi cấu hình được sửa lỗi, một commit mới đã được push lên nhánh `develop_2.0`, và GitHub Actions đã bắt đầu một đợt triển khai mới.



Hình 8: Kiểm tra và khắc phục lỗi triển khai Elastic Beanstalk

5.4.17 Bước 17: Khôi phục (Roll Back) về Phiên bản Ổn định

Nếu một đợt triển khai mới gặp sự cố và ảnh hưởng đến môi trường đang chạy, bạn có thể triển khai lại một phiên bản ứng dụng ổn định trước đó.

Việc khôi phục (rollback) được thực hiện từ:

Elastic Beanstalk → Application versions → Chọn một phiên bản ổn định (Select a stable version) → Deploy

Rollback cho phép trang web tiếp tục hoạt động bình thường trong khi phiên bản mới đang được điều chỉnh và khắc phục lỗi.

5.4.18 Bước 18: Dọn dẹp Tài nguyên Không sử dụng

Các tài nguyên không sử dụng cần được rà soát thường xuyên để giảm dung lượng lưu trữ và chi phí trên AWS.

Quy trình dọn dẹp có thể bao gồm:

- Xóa các Docker image cũ khỏi Amazon ECR.
- Xóa các phiên bản ứng dụng Elastic Beanstalk không sử dụng nữa.
- Xóa các deployment bundles (gói triển khai) cũ khỏi Amazon S3.
- Xóa các nhóm log (log groups) không cần thiết trên CloudWatch.
- Xóa các chính sách (policies) và roles IAM không còn dùng.

Các container chạy local có thể được dừng lại bằng lệnh:

```
docker compose down
```

Volume PostgreSQL cục bộ cũng có thể bị xóa khi bạn cần một cơ sở dữ liệu mới, sạch hoàn toàn:

```
docker compose down -v
```

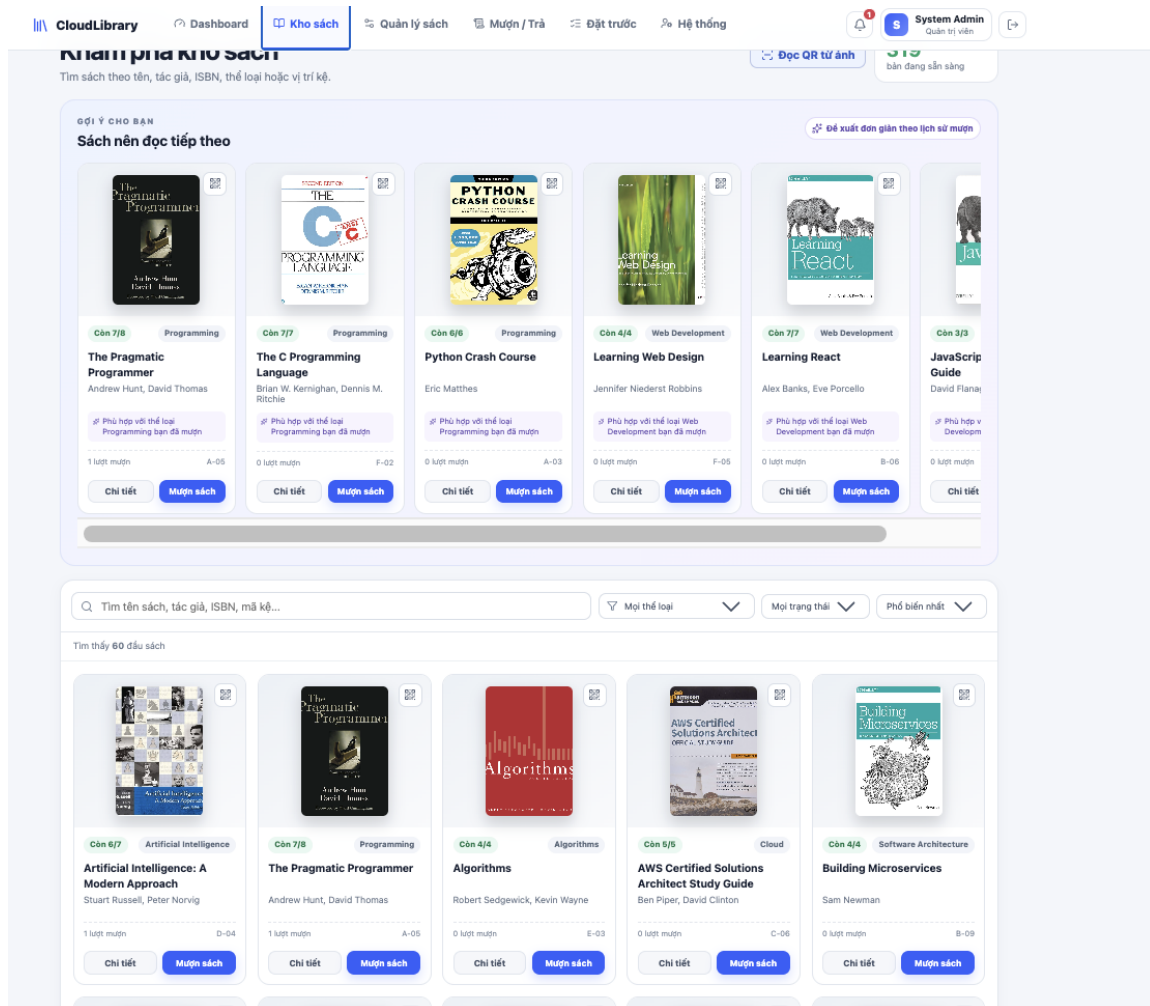
Lưu ý: Không được xóa các tài nguyên đang được ứng dụng hiện tại sử dụng.

5.5 Kết quả Triển khai

Sau khi hoàn thành quy trình triển khai, dự án đã đạt được các kết quả sau:

- Frontend và backend đã được đóng gói thành các Docker image độc lập.
- Docker images được lưu trữ an toàn trong các kho lưu trữ private của Amazon ECR.
- GitHub Actions tự động thực hiện quá trình kiểm thử và build ứng dụng.
- GitHub Actions được cấp quyền xác thực với AWS an toàn thông qua OpenID Connect.
- Mỗi lượt push code lên develop_2.0 đều kích hoạt luồng triển khai tự động.
- Elastic Beanstalk đảm nhận quản lý môi trường Docker và các phiên bản ứng dụng.
- Amazon CloudWatch thu thập đầy đủ logs triển khai, logs ứng dụng, và các metrics về cơ sở hạ tầng.
- Các đợt triển khai lỗi có thể được theo dõi và xử lý thông qua GitHub Actions, logs của Elastic Beanstalk, CloudWatch, và CloudShell.
- Ứng dụng có thể được roll back về một phiên bản ổn định trước đó bất cứ khi nào cần thiết.

Hội thảo này đã chứng minh tính hiệu quả của một luồng triển khai (deployment pipeline) thực tế dành cho một ứng dụng full-stack được container hóa. Quy trình này bắt đầu từ việc kiểm chứng cục bộ và tiếp tục xuyên suốt qua GitHub Actions, Amazon ECR, Elastic Beanstalk, và Amazon CloudWatch. Nhờ đó, quy trình triển khai trở nên có khả năng lặp lại (repeatable), dễ dàng truy xuất nguồn gốc (traceable) và gần như được tự động hóa hoàn toàn.



Hình 9: CloudLibrary hoạt động trên môi trường Elastic Beanstalk public

6 Tự đánh giá

Trong quá trình thực tập tại Công ty TNHH Amazon Web Services Việt Nam từ ngày 08/06/2026 đến 30/07/2026, tôi đã có cơ hội học hỏi, thực hành và áp dụng những kiến thức đã học ở trường vào môi trường làm việc thực tế.

Tôi đã tham gia **phát triển frontend của hệ thống CloudLibrary và tích hợp với cơ sở hạ tầng đám mây AWS**, qua đó cải thiện các kỹ năng **lập trình React, DevOps (Docker, CI/CD), giải quyết vấn đề và giao tiếp giữa các nhóm**.

Về thái độ làm việc, tôi luôn cố gắng hoàn thành tốt các nhiệm vụ được giao, tuân thủ các quy định tại nơi làm việc và tích cực trao đổi với đồng nghiệp để nâng cao hiệu quả công việc.

Để phản ánh một cách khách quan về kỳ thực tập của mình, tôi xin tự đánh giá bản thân dựa trên các tiêu chí sau:

STT	Tiêu chí	Mô tả	Tốt	Khá	TB
1	Kiến thức & kỹ năng chuyên môn	Hiểu biết về lĩnh vực, áp dụng kiến thức vào thực tế, sử dụng thành thạo các công cụ, chất lượng công việc	✓		
2	Khả năng học hỏi	Khả năng tiếp thu kiến thức mới và học hỏi nhanh chóng	✓		
3	Sự chủ động	Chủ động trong công việc, tìm kiếm nhiệm vụ mà không cần đợi phân công	✓		
4	Tinh thần trách nhiệm	Hoàn thành công việc đúng hạn và đảm bảo chất lượng	✓		
5	Tính kỷ luật	Tuân thủ thời gian, nội quy và các quy trình làm việc	✓		
6	Tinh thần cầu tiến	Sẵn sàng đón nhận các ý kiến phản hồi và cải thiện bản thân	✓		
7	Giao tiếp	Trình bày ý tưởng và báo cáo công việc một cách rõ ràng		✓	
8	Làm việc nhóm	Phối hợp hiệu quả với đồng nghiệp và tích cực tham gia vào nhóm	✓		
9	Tác phong chuyên nghiệp	Tôn trọng đồng nghiệp, đối tác và môi trường làm việc	✓		
10	Kỹ năng giải quyết vấn đề	Nhận diện vấn đề, đề xuất giải pháp và thể hiện sự sáng tạo		✓	
11	Đóng góp cho dự án/nhóm	Hiệu quả công việc, đóng góp ý tưởng đổi mới, được nhóm ghi nhận	✓		
12	Đánh giá chung	Đánh giá tổng thể toàn bộ quá trình thực tập	✓		

Các điểm cần cải thiện

- củng cố tính kỷ luật và tuân thủ nghiêm ngặt các nội quy, quy định của công ty hoặc bất kỳ tổ chức nào.
- Cải thiện tư duy giải quyết vấn đề.

- Nâng cao kỹ năng giao tiếp trong các tương tác hàng ngày cũng như trong môi trường công việc chuyên nghiệp, bao gồm cả kỹ năng xử lý tình huống hiệu quả.

7 Chia sẻ và Phản hồi

7.1 Đánh giá Chung

- **Môi trường làm việc:** Môi trường tại AWS Việt Nam vô cùng chuyên nghiệp, đổi mới và thân thiện. Các thành viên trong chương trình First Cloud AI Journey (FCAJ) rất nhiệt tình hỗ trợ và luôn sẵn lòng chia sẻ kiến thức chuyên môn. Không gian làm việc truyền cảm hứng và được trang bị hoàn hảo, giúp tôi duy trì sự tập trung và năng suất cao trong suốt kỳ thực tập.
- **Sự hỗ trợ từ Mentor / Đội ngũ Admin:** Mentor của tôi đã hướng dẫn vô cùng tận tình, đặc biệt là khi tôi phải tiếp cận các khái niệm đám mây phức tạp như Docker, ECR và pipeline CI/CD. Anh/chị ấy khuyến khích tôi tự giải quyết vấn đề nhưng vẫn luôn sẵn sàng hỗ trợ khi tôi gặp khúc mắc. Đội ngũ admin cũng rất tuyệt vời, đảm bảo quá trình tiếp nhận (onboarding) diễn ra suôn sẻ và cung cấp kịp thời mọi tài nguyên cần thiết.
- **Mức độ phù hợp của công việc với chuyên ngành:** Việc xây dựng frontend cho dự án CloudLibrary hoàn toàn phù hợp với các môn học về công nghệ phần mềm của tôi ở trường đại học. Hơn nữa, công việc này còn giúp tôi làm quen với cơ sở hạ tầng đám mây hiện đại và các phương pháp DevOps, thu hẹp khoảng cách giữa lý thuyết học thuật và tiêu chuẩn thực tế của ngành.
- **Cơ hội học tập & Phát triển kỹ năng:** Tôi đã thu nhận được rất nhiều kỹ năng thực tế trong giai đoạn này. Về mặt kỹ thuật, tôi đã nâng cao trình độ về React, Vite và container hóa. Về mặt chuyên môn, tôi đã phát triển được các kỹ năng mềm quan trọng như thích ứng với quy trình làm việc Agile, giao tiếp hiệu quả trong môi trường doanh nghiệp và giải quyết vấn đề có cấu trúc.
- **Văn hóa công ty & Tinh thần đồng đội:** Văn hóa ở đây cực kỳ tích cực và hòa nhập. Mọi người đều có tinh thần làm chủ cao, tôn trọng lẫn nhau và hợp tác tốt. Dù chỉ là thực tập sinh, những đóng góp của tôi vẫn được ghi nhận và trân trọng, điều này khiến tôi cảm thấy mình là một phần không thể thiếu của đội ngũ.
- **Chính sách / Quyền lợi thực tập:** Chương trình hỗ trợ thực tập sinh rất tốt, cung cấp điều kiện làm việc linh hoạt cùng cơ hội tiếp cận vô giá với các buổi đào tạo nội bộ và tài nguyên của AWS. Những quyền lợi này đã đẩy nhanh đáng kể quá trình học hỏi của tôi.

7.2 Các câu hỏi bổ sung

- **Điều gì khiến bạn cảm thấy hài lòng nhất trong kỳ thực tập?**

Việc triển khai thành công frontend của CloudLibrary lên AWS Elastic Beanstalk và nhìn thấy pipeline CI/CD tự động hoạt động mượt mà là khoảnh khắc đáng giá nhất trong kỳ thực tập của tôi.

- **Bạn nghĩ công ty nên cải thiện điều gì cho các thực tập sinh trong tương lai?**

Chương trình FCAJ vốn dĩ đã được cấu trúc rất bài bản và toàn diện. Tôi đánh giá cao định dạng hiện tại và chỉ hy vọng đội ngũ sẽ duy trì tiêu chuẩn cao này cũng như tiếp tục cung cấp các dự án thực tiễn cho các lứa thực tập sinh tiếp theo.

- **Nếu giới thiệu cho một người bạn, bạn có khuyên họ thực tập ở đây không? Tại sao có hoặc tại sao không?**

Chắc chắn rồi. Tôi đặc biệt khuyên các bạn nên tham gia chương trình này vì đây là cơ hội tuyệt vời để làm việc với các công nghệ đám mây tiên tiến, học hỏi trực tiếp từ các chuyên gia trong ngành và đóng góp vào các dự án có ý nghĩa.

7.3 Đề xuất & Kỳ vọng

- **Đề xuất để cải thiện trải nghiệm thực tập:**

Phương pháp học tập qua dự án (như việc xây dựng CloudLibrary) vô cùng hiệu quả. Tiếp tục giao cho thực tập sinh các dự án thực tế toàn diện (end-to-end) là cách tốt nhất để thúc đẩy sự phát triển.

- **Bạn có muốn tiếp tục tham gia chương trình này trong tương lai không?**

Có, tôi rất mong đợi cơ hội được tham gia các chương trình nâng cao hoặc tiếp tục hành trình phát triển nghề nghiệp của mình cùng AWS trong tương lai.

- **Các ý kiến đóng góp khác:**

Tôi muốn bày tỏ lòng biết ơn sâu sắc nhất tới mentor của mình và toàn thể đội ngũ FCAJ vì sự hỗ trợ không ngừng nghỉ của mọi người, giúp kỳ thực tập này trở thành một trải nghiệm thực sự mang tính bước ngoặt!